KQL Detection Engineering - Reference & Challenges

A working reference for hunting, parsing, correlation, and time-series detection in Microsoft Sentinel / ADX, with 20 graded challenges.

`Audience: SOC / DFIR  Â·  Scope: Sysmon, SecurityEvent, DeviceEvents, custom tables  Â·  Light-mode, print-ready, single file.`

CONTENTS

1. Pipeline model & flow control

KQL is a left-to-right pipeline: each tabular operator consumes the row-set produced by the previous one. Order matters for both correctness and cost - operators that reduce row count or column width should run first.

let, materialize, toscalar

`let` binds an expression, table, function, or scalar. `materialize()` caches a tabular subquery so it isn't re-executed each time it's referenced. `toscalar()` converts a 1Ã—1 result into a scalar usable in `where` and other operators.

```kql
let Lookback = ago(7d);
let AdminUsers = dynamic(["svc-backup", "administrator", "da-bryce"]);
let Recent4625 = materialize(
    SecurityEvent
    | where TimeGenerated > Lookback
    | where EventID == 4625
);
let TotalFails = toscalar(Recent4625 | count);
Recent4625
| summarize n = count() by Account
| extend share = round(100.0 * n / TotalFails, 2)
| order by n desc
```

PITFALL Without `materialize()`, a `let`-bound subquery referenced multiple times is recomputed each time. With it, the result is held in memory once. Use it for any reused subquery, but watch memory: the cached set must fit.

Functions (let-bound)

```kql
// A reusable parameterized function
let FailedLogonsByUser = (lookback:timespan, user:string) {
    SecurityEvent
    | where TimeGenerated > ago(lookback)
    | where EventID == 4625
    | where Account =~ user
    | project TimeGenerated, Computer, Account, IpAddress, FailureReason, LogonType
};
FailedLogonsByUser(24h, "CORP\\bryce")
```

VS. MICROSOFT DOCS

Microsoft's reference shows `let` only as a name binding. In practice, treat your `let` block as a small standard library at the top of every saved hunt - function definitions, named constants (lookback windows, suppression lists), and materialized subqueries - so the body of the query is the actual logic.

2. Filtering & shaping

String predicates, ranked by cost (cheapest first)

Operator	Indexed?	When to reach for it	Example
`==` / `=~`	Yes (exact)	Known full value; `=~` for case-insensitive	`EventID == 4624`
`has` / `has_cs`	Yes (term)	Whole-term match, anywhere in field	`CommandLine has "rundll32"`
`has_any` / `has_all`	Yes	Multi-term OR / AND	`CommandLine has_any (lolbins)`
`startswith` / `endswith`	Partial	Prefix / suffix	`FileName endswith ".dll"`
`contains`	No (scan)	Substring inside a term	`Image contains "syswow"`
`matches regex`	No (scan)	Pattern match - last resort	`cmd matches regex @"\\b[a-f0-9]{32}\\b"`
`in` / `in~` / `!in`	Yes	Membership against literal or dynamic set	`EventID in (4624,4625,4634)`

> **RULE OF THUMB** `has` beats `contains` by an order of magnitude on indexed columns. If you're tempted to write `contains "powershell"`, try `has "powershell"` first and only fall back to `contains` when the term is glued to other characters (`"powershell.exe"` still works with `has` because it's tokenized; `"poweshellxx"` would not).

The project family

```
DeviceProcessEvents
| project        TimeGenerated, DeviceName, ProcessCommandLine, AccountName     // keep only
| project-keep   "Time*", DeviceName                                            // keep matc
| project-away   ProcessId, InitiatingProcessId                                 // drop thes
| project-rename Host = DeviceName, User = AccountName                          // rename in
| project-reorder TimeGenerated, Host, User, ProcessCommandLine                 // reorder, k
```

extend - derive columns

```
SecurityEvent
| where EventID == 4688
| extend
    HourOfDay     = datetime_part("hour", TimeGenerated),
    IsAfterHours  = iff(datetime_part("hour", TimeGenerated) !between (7 .. 19), true, false
    ParentLeaf    = tostring(split(ParentProcessName, "\\")[-1]),
    ChildLeaf     = tostring(split(NewProcessName,    "\\")[-1]),
    UnusualParent = iff(ChildLeaf =~ "powershell.exe" and ParentLeaf in~ ("winword.exe","exc
| where UnusualParent
```

distinct vs. summarize-by

Use `distinct` when you want unique combinations and nothing else. Use `summarize ... by` when you also need an aggregate. `distinct` is implemented as a shuffle and is generally fine, but on huge cardinality it can be expensive - consider `summarize hint.shufflekey=Key by Key`.

```
DeviceProcessEvents | distinct DeviceName, AccountName

// equivalent in row count, but you can also keep an aggregate cheaply:
DeviceProcessEvents | summarize firstSeen = min(TimeGenerated) by DeviceName, AccountName
```

3. Parsing & type coercion

parse - declarative pattern parsing

```
// Sysmon EID 1 (Process Create) is often shipped as a single Message blob
Event
| where Source == "Microsoft-Windows-Sysmon" and EventID == 1
| parse EventData with * "<Data Name='UtcTime'>" UtcTimeRaw "</Data>"
                        * "<Data Name='Image'>"    Image      "</Data>"
                        * "<Data Name='CommandLine'>" CommandLine "</Data>"
                        * "<Data Name='ParentImage'>"  ParentImage "</Data>"
                        *
| extend UtcTime = todatetime(UtcTimeRaw)
| project UtcTime, Computer, Image, ParentImage, CommandLine
```

> **TIP** The `*` in a `parse` pattern is "skip everything up to the next literal." Use `parse-where` to discard rows that don't match instead of returning empty strings, and `parse kind=regex` for regex captures. Use `parse-kv` for `key=value, key=value` blobs (firewall logs, Linux audit, etc.).

parse-kv - when log lines are key=value soup

```
Syslog
| where ProcessName == "sshd"
| parse-kv SyslogMessage as (user:string, rhost:string, port:int) with (pair_delimiter=" ",
| project TimeGenerated, Computer, user, rhost, port
```

extract / extract_all - regex captures

```
DeviceNetworkEvents
| where RemoteUrl matches regex @"https?://[^/]+"
| extend Host = extract(@"https?://([^/:]+)", 1, RemoteUrl)
| extend AllIPs = extract_all(@"\b(?:\d{1,3}\.){3}\d{1,3}\b", RemoteUrl)
| project TimeGenerated, DeviceName, RemoteUrl, Host, AllIPs
```

parse_json & dynamic paths

```
DeviceEvents
| where ActionType == "AsrLsassCredentialTheftAudited"
| extend Add = parse_json(AdditionalFields)
| extend
    TargetImage  = tostring(Add.TargetImageFileName),
    SourceCmd    = tostring(Add.SourceCommandLine),
    Hashes       = tostring(Add.SHA256)
| project TimeGenerated, DeviceName, TargetImage, SourceCmd, Hashes
```

> **GOTCHA** Dynamic columns are loosely typed. `tostring`, `tolong`, `todatetime` on JSON-extracted values is not optional - without it, downstream `where` clauses will silently fail to match because you're comparing string vs dynamic.

split, strcat, replace_string, replace_regex

```
| extend PathParts  = split(FolderPath, "\\")
| extend Drive      = tostring(PathParts[0])
| extend FileName   = tostring(PathParts[-1])
| extend Cleaned    = replace_regex(CommandLine, @"\s+", " ")
| extend Anonymized = replace_string(Cleaned, "CORP\\", "REDACTED\\")
| extend Joined     = strcat(DeviceName, "::", Image, "::", AccountName)
```

Type coercion

From â†’ To	Function	Returns null on bad input?
any â†’ datetime	`todatetime()`	Yes
any â†’ timespan	`totimespan()`	Yes
string â†’ int/long/double	`toint()` / `tolong()` / `todouble()`	Yes
string â†’ bool	`tobool()`	Yes
any â†’ string	`tostring()`	No (returns "")
any â†’ dynamic	`todynamic()` / `parse_json()`	Yes
hex string â†’ long	`tolong(strcat("0x", h))` trick	If malformed

4. Time, bins & date math

The time toolkit

```
// Common building blocks
| extend Bucket5m    = bin(TimeGenerated, 5m)             // floor to 5-minute boundary
| extend AlignedHour = bin_at(TimeGenerated, 1h, datetime(2026-01-01 00:00))  // align to a
| extend DayStart    = startofday(TimeGenerated)
| extend DayOfWeek   = dayofweek(TimeGenerated)         // 0d == Sunday, 6d == Saturday
| extend AgeMinutes  = 1.0 * (now() - TimeGenerated) / 1m
| extend Pretty      = format_datetime(TimeGenerated, "yyyy-MM-dd HH:mm:ss.fff")
```

Sliding windows with bin

```
// Per-host login attempts in 1-minute buckets, last 24h
SecurityEvent
| where TimeGenerated > ago(24h) and EventID in (4624,4625)
| summarize
    Success = countif(EventID == 4624),
    Failed  = countif(EventID == 4625)
  by Computer, bin(TimeGenerated, 1m)
| order by Computer asc, TimeGenerated asc
```

Format tokens that bite

Token	Meaning	Token	Meaning
`yyyy`	4-digit year	`HH`	24-hour hour
`MM`	2-digit month	`hh`	12-hour hour
`dd`	day-of-month	`mm`	minute
`fff`	milliseconds	`ss`	second
`ddd`	abbrev. weekday	`tt`	AM/PM

> **PITFALL** Sysmon's `UtcTime` field is a string, not a datetime. Always wrap it: `todatetime(UtcTime)`. Worse, on some collectors the value is rendered with a trailing space - `trim_end()` first.

5. Aggregation & `summarize`

Core aggregators

Aggregator	Use	Notes
`count()` / `countif(pred)`	Row counts, conditional counts	`countif` avoids splitting into multiple summarize passes
`dcount()` / `dcountif()`	Approx distinct count (HLL)	4th param sets accuracy 0..4 (3 â‰ˆ 0.8% error, default 1)
`count_distinct()`	Exact distinct	More expensive; prefer `dcount` at scale
`arg_max(x, *)` / `arg_min`	Row with max/min of x; `*` keeps all cols	Replaces `top 1 by` in grouped queries
`make_list(x [, n])`	Ordered list (preserves input order in serialized streams)	Optional cap; combine with `serialize` for guaranteed order
`make_set(x [, n])`	Deduplicated unordered set	Cheaper than `make_list` for cardinality questions
`make_bag(d [, n])`	Merge dynamic property-bags	Pair with `bag_unpack`
`percentile(x, p)` / `percentiles(x, p1, p2, ...)`	Latency analysis, anomaly thresholds	Use `percentiles_array` when feeding to series functions
`hll(x)` / `tdigest(x)`	Composable distinct counts / quantiles	Aggregate per shard, then `hll_merge` / `tdigest_merge`

arg_max - the operator you'll use weekly

```
// Latest known IP per device (and any other column you want to keep)
DeviceNetworkInfo
| summarize arg_max(TimeGenerated, *) by DeviceName
| project DeviceName, IPAddresses, MacAddress, NetworkAdapterName, TimeGenerated
```

make_list with order - the serialize trick

```
// Per-host process tree in order of appearance
DeviceProcessEvents
| where TimeGenerated > ago(1h)
| order by DeviceName asc, TimeGenerated asc
| serialize
| summarize ProcessChain = make_list(FileName, 100) by DeviceName
```

WHY SERIALIZE? `summarize` on a parallelized stream does not guarantee input order inside `make_list`. `order by ... | serialize` forces a single ordered stream feeding the aggregator. Without it, the chain may be shuffled.

Pivots with bag_unpack

```
SecurityEvent
| where TimeGenerated > ago(7d) and EventID in (4624,4625,4634,4672,4688)
| summarize n = count() by Computer, EventID
| extend packed = bag_pack(tostring(EventID), n)
| summarize bag = make_bag(packed) by Computer
| evaluate bag_unpack(bag)
```

6. Joins, lookups, unions

Join kinds at a glance. `innerunique` *is the default - and the source of most surprise duplicates / missing rows.*

The eight kinds, with the trap explained

kind=	What it returns	Default?
`innerunique`	Left de-duplicated on key, then inner-joined to right. Fast, but means duplicate *left* rows are silently dropped.	Yes - this is the default if you omit `kind=`.
`inner`	Cartesian of all matching pairs. The "SQL inner join" most analysts expect.	No
`leftouter`	All L, matched R or `null`.	No
`rightouter`	All R, matched L or `null`.	No
`fullouter`	Both, with nulls on either side.	No
`leftanti`	L rows with no match in R. **Use for "first-time-seen".**	No
`rightanti`	R rows with no match in L.	No
`leftsemi` / `rightsemi`	Like inner, but returns only the L (or R) columns. Cheaper than projecting away after a join.	No

> **DEFAULT TRAP** `T1 | join T2 on Key` is `innerunique`. If T1 has 100 rows and 10 share a key, you'll get back fewer rows than you'd expect from a SQL inner join. *Always specify* `kind=` *explicitly* for production rules.

The "first time seen" idiom (leftanti)

```
// Process executions in the last hour that did NOT appear anywhere in the prior 30 days
let Baseline = DeviceProcessEvents
    | where TimeGenerated between (ago(30d) .. ago(1h))
    | distinct DeviceName, FileName, SHA256;
DeviceProcessEvents
| where TimeGenerated > ago(1h)
| join kind=leftanti Baseline on DeviceName, FileName, SHA256
| project TimeGenerated, DeviceName, FileName, SHA256, ProcessCommandLine, AccountName
```

lookup - optimized one-sided join

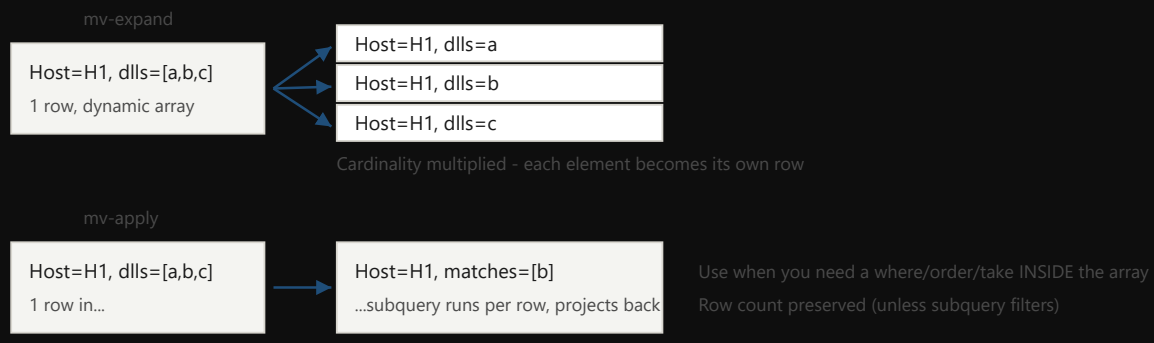
```
let AdminUsers = externaldata(SamAccountName:string, IsAdmin:bool)
    [@"https://watchlist.example/admins.csv"] with(format="csv", ignoreFirstRecord=true);
SecurityEvent
| where EventID == 4624
| lookup kind=leftouter AdminUsers on $left.Account == $right.SamAccountName
| where IsAdmin == true
```

union - across tables and workspaces

```
union isfuzzy=true
    (SecurityEvent | where EventID == 4624 | project TimeGenerated, Computer, Account, IpAdd
    (SigninLogs     | project TimeGenerated, Computer = DeviceDetail.deviceId, Account = Use
| summarize n = count() by _Source, Account
| order by n desc
```

HINT When one side is small and one is huge, hint the join: `| join hint.strategy=broadcast SmallTable on Key`. When both are large but well-keyed, `hint.shufflekey=Key` distributes the work.

7. Multi-value: `mv-expand` vs `mv-apply`



`mv-expand` explodes one row into many. `mv-apply` runs a subquery per row over the array and projects the result back - preserving the parent row.

mv-expand

```
DeviceProcessEvents
| where ProcessCommandLine has "powershell"
| extend Tokens = split(ProcessCommandLine, " ")
| mv-expand Token = Tokens
| where Token matches regex @"^[A-Za-z0-9+/=]{200,}$"          // long base64-ish tokens
| project TimeGenerated, DeviceName, ProcessCommandLine, Token
```

mv-apply - when you need where/order inside the array

```
// Per host: keep only DLL load events whose ImageLoaded matches your watch-list
let watch = dynamic(["samlib.dll","vaultcli.dll","wdigest.dll"]);
DeviceImageLoadEvents
| where TimeGenerated > ago(1d)
| summarize Loads = make_list(pack("t", TimeGenerated, "dll", FileName)) by DeviceName, Init
| mv-apply Load = Loads to typeof(dynamic) on (
    where tostring(Load.dll) in~ (watch)
    | summarize Hits = make_list(Load)
  )
| where array_length(Hits) >= 2             // at least 2 of the watched DLLs in this batch
| project DeviceName, InitiatingProcessFileName, Hits
```

WHEN TO CHOOSE WHICH Use `mv-expand` when you want a row per array element and don't care about the parent. Use `mv-apply` when you want to filter / sort / aggregate inside the array but keep one row per parent. `mv-apply` is also where you do "first N elements matching predicate," which is otherwise awkward.

pack / bag_unpack - building and flattening property bags

```
| extend Evt = pack("id", EventID, "src", IpAddress, "acct", Account)
| summarize Events = make_list(Evt) by Computer
// elsewhere, to flatten a single-row property bag back into columns:
| evaluate bag_unpack(Properties)
```

8. Advanced: `partition`, `evaluate`, `serialize`

partition - run a sub-pipeline per key

```
// For each device, keep only the top 5 commandlines by frequency in the last day.
DeviceProcessEvents
| where TimeGenerated > ago(1d)
| partition hint.strategy=native by DeviceName (
    summarize Hits = count() by ProcessCommandLine, DeviceName
    | top 5 by Hits
)
```

strategy	When to use
`hint.strategy=native`	Best when keys partition naturally across the cluster (e.g. tenant id, device id with high cardinality). Cheapest at scale.
`hint.strategy=shuffle`	When data isn't already partitioned by key; forces a redistribute.
`hint.strategy=legacy`	Default; processes per-key serially. Fine for low cardinality.

evaluate plugins - the power tools

Plugin	Purpose
`autocluster`	Finds the smallest set of column patterns that cover most of the rows. Triage gold for noisy alert tables.
`basket`	Like autocluster but exhaustively enumerates frequent itemsets above a min-support threshold.
`diffpatterns`	Compares two sub-populations (e.g. anomaly vs baseline) and surfaces columns that differentiate them.
`pivot`	Excel-style pivot: turn distinct values of a column into columns.
`bag_unpack`	Flattens a dynamic property bag into individual columns.
`narrow`	Inverse of pivot - collapses many columns into `(Column, Value)` pairs.
`infer_storage_schema`	Reads sample data to infer schema for `externaldata`.

```
// Triage 7 days of process create alerts: what patterns dominate?
DeviceProcessEvents
| where TimeGenerated > ago(7d) and InitiatingProcessFileName =~ "powershell.exe"
| project DeviceName, AccountName, FileName, FolderPath, ProcessCommandLine
| evaluate autocluster(0.5)         // 0.5 = cover 50% of rows with the fewest patterns
```

```
// What columns differ between "rare" and "common" parent->child relationships?
DeviceProcessEvents
| where TimeGenerated > ago(7d)
| summarize n = count() by InitiatingProcessFileName, FileName, AccountDomain, FolderPath
| extend bucket = iff(n <= 2, "rare", "common")
| evaluate diffpatterns(bucket, "rare", "common")
```

serialize and the windowing functions

Once a stream is serialized (single ordered partition), you can use row-aware functions:

`prev()`, `next()`, `row_number()`, `row_rank_min/dense()`, `row_cumsum()`,

`row_window_session()`.

```
// Inter-event interval per host
DeviceProcessEvents
| where TimeGenerated > ago(1h)
| order by DeviceName asc, TimeGenerated asc
| serialize
    PrevTime = prev(TimeGenerated, 1),
    PrevHost = prev(DeviceName,    1),
    Gap      = iff(DeviceName == prev(DeviceName, 1), TimeGenerated - prev(TimeGenerated, 1)
| where isnotnull(Gap) and Gap < 1s
| project DeviceName, TimeGenerated, FileName, Gap
```

row_window_session - sessionize without joining

```
// Group events into sessions per device when gap > 5 minutes
DeviceProcessEvents
| where TimeGenerated > ago(1d)
| order by DeviceName asc, TimeGenerated asc
| extend SessionStart = row_window_session(TimeGenerated, 5m, 5m, DeviceName != prev(DeviceN
| summarize Events = count(), Files = make_set(FileName, 100) by DeviceName, SessionStart
```

range - synthesize rows

```
// Build a continuous 1-min spine to left-join sparse counts against
range ts from ago(1d) to now() step 1m
| join kind=leftouter (
    DeviceNetworkEvents
    | where TimeGenerated > ago(1d)
    | summarize n = count() by ts = bin(TimeGenerated, 1m)
) on ts
| project ts, n = coalesce(n, 0)
| order by ts asc
```

9. Time series & the `step` parameter

make-series - vectorized time series construction

```
DeviceNetworkEvents
| where TimeGenerated > ago(7d) and ActionType == "ConnectionSuccess"
| make-series n = count() default=0
    on TimeGenerated from ago(7d) to now() step 15m
    by DeviceName, RemoteIP
```

`step` is the bucket size. `default=0` fills empty buckets with zero - critical, because gaps in the series will break `series_decompose`.

Anomalies and forecasts

```
| extend (anomalies, score, baseline) =
    series_decompose_anomalies(n, 2.0, -1, "linefit")   // thr=2Ïƒ, period=auto, fit=linefit
| extend AnomalyAt = series_iif(anomalies != 0, TimeGenerated, datetime(null))
| mv-expand TimeGenerated to typeof(datetime), n to typeof(long), anomalies to typeof(int),
| where anomalies != 0
```

Function	Use
`series_decompose`	Splits series into baseline / seasonal / residual components.
`series_decompose_anomalies`	Above + flags points exceeding Ïƒ threshold.
`series_decompose_forecast`	Above + projects forward N points.
`series_outliers`	Tukey-style outlier flagging (no seasonal model).
`series_periods_detect`	Detects dominant periodicities - useful for beaconing.
`series_fit_line` / `series_fit_2lines`	Linear regression / piecewise; returns slope, intercept, RSquare. `2lines` finds the breakpoint.
`series_fir` / `series_iir`	Filters: moving avg / exponential smoothing.
`series_pearson_correlation`	Correlation between two series - find hosts behaving alike.

Beaconing detection (period detection)

```
DeviceNetworkEvents
| where TimeGenerated > ago(1d) and RemoteIPType == "Public"
| make-series n = count() default=0
    on TimeGenerated from ago(1d) to now() step 1m
    by DeviceName, RemoteIP
| extend (periods, scores) = series_periods_detect(n, 0.05, 1200.0, 5)
| mv-expand period = periods to typeof(double), score = scores to typeof(double)
| where score > 0.8 and period between (5 .. 600)
| project DeviceName, RemoteIP, IntervalMinutes = period, Confidence = score
| order by Confidence desc
```

READING THE RESULT `series_periods_detect(series, min_period_pct, max_period, num_periods)` returns the most likely periods (in step-units) and their scores. `step 1m` means a returned period of `15` equals 15 minutes between callbacks.

10. Detection-engineering patterns

A then B within Î”t - the sequence pattern

```
// Failed logon followed by a successful logon for the same account from the same IP within
let Window = 5m;
let Fails = SecurityEvent | where EventID == 4625
    | project FailTime = TimeGenerated, Account, IpAddress, Computer;
let Wins  = SecurityEvent | where EventID == 4624
    | project WinTime  = TimeGenerated, Account, IpAddress, Computer;
Fails
| join kind=inner Wins on Account, IpAddress, Computer
| where WinTime > FailTime and WinTime - FailTime <= Window
| summarize Fails = count(), FirstFail = min(FailTime), LastWin = max(WinTime) by Account, I
| where Fails >= 5
```

Rare-process-as-service (prevalence math)

```
let Window = 14d;
let TotalHosts = toscalar(DeviceProcessEvents | where TimeGenerated > ago(Window) | summariz
DeviceProcessEvents
| where TimeGenerated > ago(Window)
| where InitiatingProcessFileName =~ "services.exe"
| summarize Hosts = dcount(DeviceName), Sample = any(ProcessCommandLine) by FileName, Folder
| extend Prevalence = 100.0 * Hosts / TotalHosts
| where Prevalence < 1.0 and Hosts between (1 .. 3)
| order by Prevalence asc
```

Stack counting for triage

```
DeviceProcessEvents
| where TimeGenerated > ago(1d) and ProcessCommandLine has_any ("-enc","-EncodedCommand","Fr
| summarize Hits = count(), Hosts = dcount(DeviceName), Sample = any(ProcessCommandLine)
    by InitiatingProcessFileName, FileName, AccountDomain
| order by Hits desc
```

Cross-process correlation (the WerFault/LSASS shape)

```
// Sysmon: WerFault.exe spawned with LSASS PID and -pr flag, followed by ProcessAccess to LS
let Window = 2m;
let WerSpawn = DeviceProcessEvents
    | where FileName =~ "WerFault.exe" and ProcessCommandLine has "-pr"
    | project SpawnTime = TimeGenerated, DeviceName, WerPid = ProcessId, WerCmd = ProcessCom
let LsassAccess = DeviceEvents
    | where ActionType == "ProcessAccessByOthers" and FileName =~ "lsass.exe"
    | extend Add = parse_json(AdditionalFields)
    | extend AccessMask = tostring(Add.GrantedAccess), SourcePid = tolong(Add.SourceProcessI
    | project AccessTime = TimeGenerated, DeviceName, SourcePid, AccessMask;
WerSpawn
| join kind=inner LsassAccess on DeviceName
| where AccessTime between (SpawnTime .. SpawnTime + Window) and SourcePid == WerPid
| project DeviceName, SpawnTime, WerCmd, AccessTime, AccessMask
```

11. Performance & idioms

Do	Don't
Filter by time first, then by indexed columns, then derive.	Don't `extend` first and filter on the derived column - defeats index use.
Use `has` on tokenized text columns.	Don't reach for `contains` or `matches regex` by reflex.
Use `=~` for case-insensitive equality.	Don't `tolower(col) == "x"` - kills indexing.
Use `countif` / `dcountif` with a predicate.	Don't run two `summarize` branches and join them.
Use `arg_max(t, *)` for "latest row per group."	Don't `top 1 by t desc` inside a `partition` if a single `summarize arg_max` works.
Use `materialize()` on subqueries referenced 2+ times.	Don't materialize huge results "just in case" - memory cost.
For "first-time-seen": `join kind=leftanti` against a baseline.	Don't `!in` against a giant dynamic list.
For unbounded distinct counts: `dcount(x, 3)` (HLL).	Don't reach for `count_distinct()` on hundreds of millions of rows.

The "filter early" rule, illustrated

```
// Bad: extend first, filter the derived column
DeviceProcessEvents
| extend Lower = tolower(ProcessCommandLine)
| where Lower has "mimikatz"
| where TimeGenerated > ago(1h)

// Good: time first, indexed-token filter, never lowercase
DeviceProcessEvents
| where TimeGenerated > ago(1h)
| where ProcessCommandLine has "mimikatz"
```

12. Sentinel-specifics

Analytics rule anatomy

Sentinel scheduled rules expect: a query that returns rows; `TimeGenerated` on each row; entity mappings (Account / Host / IP / FileHash / URL â€¦) referencing column names; optional custom details for alert fields; and a query period ≤ query frequency Ã— N (set in rule UI).

```kql
// Skeleton ready to paste into Sentinel "Custom rule wizard"
let Lookback = 1h;
DeviceProcessEvents
| where TimeGenerated > ago(Lookback)
| where FileName =~ "WerFault.exe" and ProcessCommandLine has "-pr"
| project TimeGenerated, DeviceName, AccountName, ProcessCommandLine, ProcessId, SHA256
// Entities to map in UI: Host=DeviceName, Account=AccountName, FileHash=SHA256
// Custom details: ProcessCommandLine, ProcessId
```

Watchlists & externaldata

```kql
// Built-in watchlist
let CrownJewels = _GetWatchlist("CrownJewelHosts") | project Hostname;
DeviceLogonEvents
| where TimeGenerated > ago(1h)
| where DeviceName in (CrownJewels)

// External CSV (e.g. published Tier-0 list)
let Tier0 = externaldata(Sam:string)[@"https://<<ENV>>/tier0.csv"] with(format="csv", ignore
```

Sentinel functions you'll actually use

Function	Purpose
`_GetWatchlist("Name")`	Resolves a Sentinel watchlist as a table.
`SecurityAlert` / `SecurityIncident`	Built-in alert & incident tables.
`geo_info_from_ip_address(ip)`	Built-in GeoIP enrichment (country, city, lat/long).
`iff(EventID == 4624, ...)`	Branchy column derivation; nests cleanly.
`_ResourceId`	Hidden column - useful for cross-tenant queries.

[Slot for screenshot: Sentinel "Analytics rule wizard - Set rule logic" tab showing entity mapping for a custom rule.]

Replace this slot with: `<img src="entity-mapping.png" alt="...">`

Appendix Â· Quick reference index

Operators (alphabetical)

as Â· count Â· distinct Â· evaluate Â· extend Â· facet Â· find Â· fork Â· getschema Â· invoke Â· join Â· limit/take Â· lookup Â· make-series Â· mv-apply Â· mv-expand Â· order/sort Â· parse Â· parse-kv Â· parse-where Â· partition Â· print Â· project Â· project-away Â· project-keep Â· project-rename Â· project-reorder Â· range Â· reduce Â· render Â· sample Â· sample-distinct Â· search Â· serialize Â· summarize Â· top Â· top-hitters Â· top-nested Â· union Â· where

High-leverage scalar/aggregate functions

ago Â· arg_max Â· arg_min Â· array_length Â· array_sum Â· bag_pack Â· bag_unpack Â· base64_decode_toarray Â· bin Â· bin_at Â· case Â· coalesce Â· count_distinct Â· countif Â· dcount Â· dcountif Â· datetime_part Â· extract Â· extract_all Â· format_datetime Â· geo_distance_2points Â· geo_info_from_ip_address Â· iff Â· iif Â· indexof Â· isnotempty Â· isnotnull Â· make_bag Â· make_list Â· make_list_if Â· make_set Â· make_set_if Â· materialize Â· mv-expand to typeof Â· now Â· pack Â· parse_json Â· parse_url Â· percentile Â· percentiles Â· prev Â· next Â· replace_regex Â· replace_string Â· row_cumsum Â· row_number Â· row_rank_dense Â· row_window_session Â· series_decompose Â· series_decompose_anomalies Â· series_decompose_forecast Â· series_fit_2lines Â· series_fit_line Â· series_pearson_correlation Â· series_periods_detect Â· set_difference Â· set_has_element Â· set_intersect Â· set_union Â· split Â· startofday Â· strcat Â· strcat_array Â· substring Â· todatetime Â· todynamic Â· toint Â· tolong Â· toreal Â· toscalar Â· tostring Â· totimespan Â· trim

[Slot for cheat sheet image: KQL operator pipeline diagram (left-to-right). Replace with your own diagram or screenshot.]

End of document. To regenerate or extend: edit this single .html file. All styling, code samples, diagrams, and image slots are inline - no external dependencies. For air-gapped sharing, copy the file as-is.